

proj_readme

Final Project - SPI Master Controller Verification

Ain Shams University - Digital Design Verification - Spring 2026

This repository is what we hand to students as their **final project starting point**. It contains:

- A specification PDF (`docs/SPI_Master_Spec.pdf`) describing the DUT.
- A golden RTL implementation of the DUT (`golden_rtl/spi_master.sv`).
- A fixed verification harness (`harness/`) that students must not modify.
- A grading contract (`harness/grading_interface.md`) that specifies exactly what the submission must look like.

Students do **not** receive `faulty_rtl/` , `reference_env/` , or `grading/` .

Those live in the course-staff copy only; they are used on our side to swap in buggy DUTs and automate grading. See "Layout" below for details.

What you are being asked to build

You receive a real SPI master IP (see `docs/SPI_Master_Spec.pdf`) and must produce a SystemVerilog verification environment that:

1. Drives an APB master into the DUT to configure it (register writes/reads).
2. Models one or more SPI slaves attached to MISO/MOSI/SCLK/SS_n[3:0].
3. Predicts the correct behaviour for every APB and SPI transaction and reports a `[SCOREBOARD_ERROR]` whenever the DUT deviates from the spec.
4. Covers the full configuration space: 4 SPI modes x {MSB/LSB-first} x {8, 16, 32}-bit x a meaningful range of `CLK_DIV` and `DELAY` values x every interrupt source.
5. Contains at least 5 SystemVerilog assertions bound to the DUT.

Constrained-random stimulus, functional coverage, and SVA are **mandatory**. UVM is **optional** and earns bonus points; UVM RAL earns further bonus.

The grader will run your regression against a version of the DUT that has one of 30 seeded bugs active at a time, and will count how many of those

bugs your environment flags. See `harness/grading_interface.md` for the exact contract.

Layout (what students get)

```
final_project/
  README.md          ← this file
  docs/
    SPI_Master_Spec.pdf    ← read this first
    generate_spec_pdf.py   ← reproducible source for the PDF
  golden_rtl/
    spi_core.sv          ← SPI shift engine sub-block (visible u_core)
    apb_regfile.sv       ← APB slave + registers + FIFOs (visible
u_regfile)
    spi_master.sv        ← thin top that instantiates both sub-blocks
  harness/
    apb_if.sv            ← APB SystemVerilog interface
    spi_if.sv            ← SPI-side interface
    dut_wrapper.sv       ← wraps the DUT, binds interfaces
    Makefile.template    ← template Makefile to copy into your dir
    grading_interface.md  ← normative submission contract (MUST read)
    examples/sv_only/    ← ready-to-clone SV-only scaffold (no UVM)
```

The DUT is deliberately split into two sub-blocks that the testbench may probe hierarchically:

```
dut_wrapper.u_dut
+-- u_regfile : apb_regfile    (APB slave, 9 registers, TX/RX FIFOs, IRQ)
+-- u_core    : spi_core       (SPI shift FSM, SCLK/MOSI/SS_n drive)
```

External pinout is unchanged; students **MUST NOT** modify the DUT. The hierarchy is exposed so you can backdoor-peek registers and bind SVA into either sub-block (`bind u_wrap.u_dut.u_regfile ...`).

Layout (course-staff only, not distributed)

```
final_project/
  faulty_rtl/
    spi_core_buggy.sv    ← bugs scoped to the SPI core
    apb_regfile_buggy.sv ← bugs scoped to APB/regfile/FIFO/IRQ/SS
    spi_master_buggy.sv  ← thin top wrapping the _buggy sub-blocks
```

reference_env/	
spi_pkg.sv	← UVM agents + scoreboard + coverage
spi_seq_pkg.sv	← UVM sequences + tests
spi_assertions.sv	← SVA bound in tb_top
tb_top.sv	← UVM top
run.ps1	← helper to compile+run in Questa
grading/	
bug_catalog.yaml	← bug_id → detection signature mapping
bug_to_spec.md	← bug_id → spec clause + minimal stimulus
recipe	
rubric.md	← authoritative grading rubric
run_grading.py	← swap DUT, run regression, emit score

The faulty RTL and bug catalog are not made available to students. All randomisation and grading pressure is applied purely through the published spec and the public harness.

How to start

1. Read `docs/SPI_Master_Spec.pdf` end to end. It is 11 pages (one-page architecture appendix added in Rev 1.1); every requirement has a requirement-id tag like `R14` that you can cite in your test plan.
2. Read `harness/grading_interface.md`. It specifies the Makefile targets, the test names, and the log conventions the grader uses.
3. Copy `harness/Makefile.template` into your submission directory and rename it to `Makefile`. Fill in the simulator-specific sections if you are not using QuestaSim.
4. Instantiate the provided `dut_wrapper` in your testbench. Do not modify `harness/dut_wrapper.sv` or either interface file; the grader replaces `golden_rtl/spi_master.sv` with the faulty version and expects your TB to keep working.
5. Build a scoreboard and coverage model based on the spec, not on reading the golden RTL. The faulty RTL's bugs were picked specifically to hit behaviours that are spec-observable but only covered by well-crafted directed + random tests.
6. Write at least the 10 mandatory tests listed in `harness/grading_interface.md`. Add as many more as you like within the 10000-run cap.

A good order of attack is: APB register access -> single-byte mode-0 transfer -> mode crossing -> width crossing -> FIFO stress -> interrupts/W1C -> CLK_DIV corner cases -> loopback -> DELAY behaviour -> error injection.

Deliverables

Submit a single zip named `<name>_<id>.zip` . It must unpack to a top-level directory named `<name>_<id>/` that follows the layout in `harness/grading_interface.md` Section 1, and must include:

- `Makefile` inheriting the template's interface
- `tb/` , `env/` , `tests/` , `sequences/` , `assertions/` directories
- `docs/test_plan.pdf` - your test plan with covergroup + SVA inventory
- `docs/final_report.pdf` - bugs found, coverage numbers, lessons learned
- `docs/coverage_report.pdf` - the GUI export of your coverage results
- `README.md` - your simulator version and any non-default flags

The grader runs (roughly) this sequence inside your submission tree:

```
make clean
make compile DUT_SRCS=<spi_core_buggy.sv> <apb_regfile_buggy.sv>
<spi_master_buggy.sv>
make regress DUT_SRCS=" ... " REGRESSION_SEEDS=<N>
make cov
make run_bonus           # only if you opted into the UVM RAL bonus
```

with `|REGRESSION_TESTS| * REGRESSION_SEEDS ≤ 10000` . Any breach of that bound zeroes the bug-detection category.

SystemVerilog-only vs UVM

UVM is **optional**. Both workflows hit the same Makefile targets and the same per-test log contract (`[TEST_PASSED]` / `[TEST_FAILED]` , `[SCOREBOARD_ERROR]` , `[ASSERTION_ERROR]`). The differences:

Area	SV-only	UVM (optional)
Top-level	<code>module tb_top</code> with <code>+TESTNAME</code>	<code>module tb_top</code> calling <code>run_test()</code>
BFMs	<code>module</code> / <code>program</code> with tasks	<code>uvm_driver</code> / <code>uvm_monitor</code>

Area	SV-only	UVM (optional)
Scoreboard	plain SV class, <code>[SCOREBOARD_ERROR]</code>	<code>uvm_scoreboard</code> with analysis ports
Coverage	<code>covergroup</code> in a class	<code>uvm_subscriber</code> with covergroup
Backdoor checks	hierarchical refs into <code>u_regfile</code> / <code>u_core</code>	<code>uvm_reg_block</code> backdoor via <code>add_hdl_path</code>
UVM bonus	not eligible	+10 if structural checks pass
RAL bonus	not eligible	+5 if <code>ral_hw_reset_test</code> passes

A ready-to-clone SV-only scaffold lives at `harness/examples/sv_only/`. Copy it, rename the test cases, and you have a submission that already hits the Makefile contract. See that directory's own `README.md` for details.

Scoring summary (see `grading/rubric.md` on the staff side)

Category	Weight
Mandatory tests pass	10
Coverage ($\geq 85\%$)	15
Assertions	5
Bugs caught (20/28)	60
Report & test plan	10
UVM bonus	+10
UVM RAL bonus	+5

Full bug-detection marks require **20 of the 28** scored bugs to be flagged. Every bug below 20 costs $60/20 = 3.0$ points on a linear ramp. Catching more than 20 does not lose points but is capped at full marks.

Note: the project originally seeded 30 bugs. Two of them (BUG_E03 / BUG_E04) were retired from scoring because their faulty values self-heal within one PCLK and are not observable from any APB stimulus path. The staff-only file `grading/bug_to_spec.md` documents the analysis. The 20/28 threshold preserves the original ~83% difficulty bar.

Rules and integrity

- You may work in teams of up to 3 people. Submit one zip per team.
 - You may share experience **between** teams but not code. Reuse of code from another team counts as academic dishonesty.
 - Do not attempt to detect the grader, short-circuit tests based on `+define+FAULTY`, or otherwise game the grading pipeline. See `harness/grading_interface.md` Section 8 for the full list of forbidden practices.
 - If you discover ambiguity in the spec, file an issue on the class forum. We will issue a spec clarification (and potentially a spec patch) that applies to every submission.
-

Acknowledgements

The SPI Master reference IP is an original teaching design created for this course. It intentionally mimics the register layout of production-grade SPI masters (e.g. the ones found in ARM MCUs) so that the verification lessons transfer directly to industry.

Good luck. Catch bugs.